DSL Compiler Project

Compiler Design Assignment — Report

Field	Details
Name	HARSITHA G P
Register No.	RA2311026050197
Class	AI & ML “A”
Subject	COMPILER DESIGN
Institution	SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

1. Project Description

This project implements a minimal but fully functional compiler for a custom Domain-Specific Language (DSL). The compiler is built entirely in C using Flex for lexical analysis and Bison for parsing. It demonstrates all five classic phases of a compiler pipeline in a clean, well-structured codebase.

Supported DSL Features:

Feature	Example
Integer variable assignment	x = 5;
Addition	z = x + y;
Subtraction	z = x - y;
Multiplication	z = x * y;
Division	z = x / y;
Parenthesised expressions	z = (x + y) * 2;
Unary minus	z = -x;
Single-line comments	# this is a comment
Multiple statements	Any number of assignments

2. Tools Used

Tool	Version	Purpose
GCC	11.4	C compiler — compiles all .c source files
Flex	2.6.4	Lexical analyser generator (tokeniser)
Bison	3.8.2	Parser generator (LALR grammar)
Make	4.3	Build automation
Ubuntu	22.04	Development environment

3. Compiler Architecture

The compiler follows the classic five-phase pipeline. Each phase is implemented as an independent C module, connected through the AST data structure:

Phase	Module	Input	Output
1. Lexical Analysis	lexer.l	Source .dsl file	Token stream
2. Parsing	parser.y	Token stream	AST root node
3. AST Construction	ast.c	Grammar actions	Tree of ASTNode
4. Semantic Analysis	semantic.c	AST root	Validated AST / errors
5. Code Generation	codegen.c	Validated AST	Three-address code

4. Module Explanations

lexer.l — Lexical Analyser

Written in Flex, this module scans the raw source text and breaks it into tokens such as NUMBER, IDENTIFIER, ASSIGN, PLUS, MINUS, TIMES, DIVIDE, SEMI, LPAREN, and RPAREN. Whitespace and # comments are silently discarded. Unknown characters trigger an immediate error with the line number.

parser.y — Parser & Grammar

Written in Bison, this module defines the LALR(1) grammar. Operator precedence (* / before + -) is declared using %left directives. Each grammar action calls an AST constructor function, building the syntax tree bottom-up. The finished root node is stored in the global ast_root variable.

ast.c / ast.h — Abstract Syntax Tree

Defines five node kinds: NODE_NUMBER (integer constant), NODE_IDENTIFIER (variable reference), NODE_ASSIGN (assignment statement), NODE_BINOP (binary operation), NODE_UNARY (unary minus), and NODE_STMTLIST (ordered list of statements). Provides constructor helpers, a recursive pretty-printer for debugging, and a recursive memory-freeing function.

semantic.c — Semantic Analysis

Performs a single-pass walk over the AST using a hash-map symbol table. For every assignment x = expr, the RHS is checked first, then x is added to the symbol table. Any identifier used in an expression that is not yet in the table generates a 'used before assignment' error. Compilation is aborted if any errors are found.

codegen.c — Intermediate Code Generation

A recursive post-order walk that emits three-address code (TAC). Binary operations produce a fresh temporary variable (t0, t1, t2, ...) and emit an instruction like: t0 = x + y. Assignments copy the result into the named variable. The TAC output is suitable as input to a further optimisation or machine-code generation back-end.

main.c — Driver

Opens the source file, calls yyparse() to trigger lexing and parsing, then chains semantic analysis and code generation. Prints a banner before each phase for clear, readable output. Optionally writes the TAC output to a file if a second argument is provided.

5. Project Folder Structure

```
Compiler-Design-DSL_RegNo_Name/
src/
lexer.l # Flex lexical analyser
parser.y # Bison grammar
ast.h / ast.c # AST node definitions
semantic.c # Semantic analysis
codegen.c # Code generation
main.c # Driver
test/
test1.dsl # Basic arithmetic
test2.dsl # Complex expressions
test3_error.dsl # Semantic error test
output/
test1.tac
test2.tac
docs/
DSL_Compiler_Report.pdf
Makefile
README.md
```

6. Steps to Run

Step 1 — Install dependencies

```
sudo apt-get update && sudo apt-get install gcc flex bison make -y
```

Step 2 — Build the compiler

```
make all
```

Step 3 — Run test 1

```
./build/dslc test/test1.dsl
```

Step 4 — Run test 2

```
./build/dslc test/test2.dsl
```

Step 5 — Test semantic error detection

```
./build/dslc test/test3_error.dsl
```

Step 6 — Save TAC output to file

```
./build/dslc test/test1.dsl output/test1.tac
```

Step 7 — Run all tests at once

```
make test
```

Step 8 — Clean build files

```
make clean
```

7. Sample Input and Output

Input — test/test1.dsl

```
x = 5;
y = 10;
z = x + y;
result = z * 2;
```

Output — Three-Address Code

```
; -- Three-Address Code
begin:
x = 5
y = 10
t0 = x + y
z = t0
```

```
t1 = z * 2
result = t1
end:
```

Semantic Error Detection — test/test3_error.dsl

```
a = 10;
c = a + b; # ERROR: b is not defined
```

Error output:

```
Semantic error: variable 'b' used before assignment
[Semantic] 1 error(s) found
```